

Problema CERCETĂȘI

Autor problemă:

Scurtă descriere a problemei

Problema cere calcularea unei sume de numere Stirling de tipul al doilea, cu indici într-un interval $[st, dr]$, modulo $M = 1\,000\,000\,007$. Soluția utilizează formula explicită pentru numerele Stirling și calculează suma folosind precalculări eficiente.

Noțiuni preliminare

Numerele Stirling de tipul al doilea, notate $S(n, k)$, reprezintă numărul de moduri de a partiționa o mulțime cu n elemente în exact k submulțimi nevide (blocuri). O formulă fundamentală pentru calculul acestora este:

$$S(n, k) = \frac{1}{k!} \sum_{i=0}^k (-1)^{k-i} \binom{k}{i} i^n \quad (1)$$

Această formulă se obține prin aplicarea principiului incluziune-excluziune pentru a număra partițiile.

Observații-cheie

Din formula de mai sus, observăm că:

$$S(n+1, j) = \frac{1}{j!} \sum_{i=0}^j (-1)^{j-i} \binom{j}{i} i^{n+1}$$

Produsul $S(n+1, j) \cdot \frac{1}{j!}$ apare natural în multe aplicații combinatoriale. Mai mult, seria alternantă $\sum_{j=0}^k (-1)^{k-j} \frac{1}{j!}$ are o formă recursivă simplă:

$$pref[k] = \sum_{j=0}^k (-1)^{k-j} \frac{1}{j!} = pref[k-1] + (-1)^k \cdot \frac{1}{k!}$$

cu $pref[0] = 1$.

Aceste observații ne permit să calculăm suma cerută:

$$\sum_{j=st}^{dr} S(n+1, j) = \sum_{j=st}^{dr} \frac{1}{j!} \sum_{i=0}^j (-1)^{j-i} \binom{j}{i} i^n$$

prin schimbarea ordinii de sumare și utilizarea sumelor prefix.

Soluție

După transformări algebrice, obținem formula utilizată în implementare:

$$\text{ans} = \sum_{i=st}^{dr} \frac{i^n}{i!} \cdot \left(\sum_{k=st-i}^{dr-i} \frac{(-1)^k}{k!} \right) \quad (2)$$

Implementarea urmează pașii:

1. Precalculăm factorialele și factorialele inverse modulo M pentru $i \leq n$:

$$\text{fact}[i] = i! \mod M, \quad \text{ifact}[i] = (i!)^{-1} \mod M$$

Prin formula $\text{inv}[i] = (M - \lfloor M/i \rfloor) \cdot \text{inv}[M \bmod i] \mod M$, obținem inversele modular rapid.

2. Calculăm suma prefix $\text{pref}[k] = \sum_{j=0}^k (-1)^{k-j} \cdot \text{ifact}[j]$ în $O(dr)$ folosind relația recursivă de mai sus.
3. Pentru fiecare i de la st la dr , calculăm termenul $\frac{i^n}{i!} \cdot (\text{pref}[dr-i] - \text{pref}[st-i-1])$ și îl adăugăm la răspuns.

Modularul $M = 1\,000\,000\,007$ este prim, deci toate inversele există.

Complexitate

- Precalcularea factorialelor: $O(n)$
- Calculul sumelor prefix: $O(dr)$
- Bucla principală: $O(dr - st + 1)$

Complexitatea totală este $O(n + dr)$, iar memoria este $O(n)$.

Corectitudine

Demonstrăm corectitudinea prin următoarele leme:

lemma Pentru orice $n, k \geq 0$, $S(n, k) = \frac{1}{k!} \sum_{i=0}^k (-1)^{k-i} \binom{k}{i} i^n$. emma

lemma Suma prefix $\text{pref}[k]$ calculată prin $\text{pref}[k] = \text{pref}[k-1] + (-1)^k \cdot \text{ifact}[k]$ satisface $\text{pref}[k] = \sum_{j=0}^k (-1)^{k-j} \cdot \text{ifact}[j]$. emma

lemma Termenul $(\text{pref}[dr-i] - \text{pref}[st-i-1])$ este egal cu $\sum_{k=st-i}^{dr-i} \frac{(-1)^k}{k!}$. emma]

Din lema 3 și formula finală, fiecare iterație a buclei adaugă corect termenul corespunzător lui i în sumă. Prin urmare, răspunsul final este exact suma cerută.

Implementare

```
#include <bits/stdc++.h>
#define ll long long
using namespace std;

const int NMAX = 100008;
const ll mod = 1000000007;

int n, r, st, dr;
ll fact[NMAX], ifact[NMAX], inv[NMAX];

void pre(int en) {
    fact[0] = fact[1] = 1LL;
    ifact[0] = ifact[1] = 1LL;
    inv[0] = inv[1] = 1LL;
    for (ll i = 2; i <= en; i++) {
        fact[i] = (fact[i-1] * i) % mod;
        inv[i] = (inv[mod % i] * (mod - mod / i)) % mod;
        ifact[i] = (ifact[i-1] * inv[i]) % mod;
    }
}

ll pw(ll baza, ll expo) {
    ll ans = 1LL;
    while (expo) {
        if (expo & 1) ans = (ans * baza) % mod;
        expo >>= 1;
        baza = (baza * baza) % mod;
    }
    return ans;
}

ll pref[NMAX];

void solve(string file) {
    ifstream fin(file + ".in");
    ofstream fout(file + ".out");

    fin >> n >> r >> st >> dr;
    pre(n);

    n -= r; n++;
    st -= r; st++;
    dr -= r; dr++;

    pref[0] = 1LL;
    for (ll i = 1; i <= dr; i++) {
        if (i & 1)
            pref[i] = (pref[i-1] - ifact[i] + mod) % mod;
        else
            pref[i] = (pref[i-1] + ifact[i]) % mod;
    }
}
```

```

ll ans = 0;
for (ll i = 0; i <= dr; i++) {
    ll pp = (pw(i, n) * ifact[i]) % mod, add;
    if (st - i - 1 >= 0)
        add = (pref[dr - i] - pref[st - i - 1] + mod) % mod;
    else
        add = pref[dr - i];

    add = (add * pp) % mod;
    ans = (ans + add) % mod;
}
fout << ans << "\n";
fin.close();
fout.close();
}

int main() {
    solve("cercetasi");
    return 0;
}

```

Algoritmul prezentat asigură un calcul corect și eficient al sumei de numere Stirling în intervalul cerut, folosind tehnici clasice de precalculare factorială și formule de incluziune-excluziune.